



Extending Harbinger – Sequence Anomalies & Range- Aware Evaluation

Collective Anomaly Detection

Point vs. Collective Anomaly

A **point anomaly** is a single observation that significantly deviates from the norm. In contrast, a **collective anomaly** is a contiguous subsequence where individual points might not be extreme, but the overall pattern or sequence is anomalous.

Why Intervals Matter

Simply identifying a single "anomalous point" within a collective anomaly loses crucial information about its duration and extent. A robust detector should return the full anomalous interval, providing a more comprehensive understanding of the event.

Rolling Median Baseline

To detect anomalies effectively, we establish a dynamic baseline using a **rolling median**. This approach is more robust than a global mean, as it tracks local trends and is less susceptible to individual outliers.

Local MAD for Variability

The **Median Absolute Deviation (MAD)** measures local variability, providing a robust alternative to standard deviation. It's used to normalize the deviation score:

$$z = \frac{|y - \text{median}|}{\text{MAD}}.$$

Run-length Filtering

To reduce false positives from transient spikes, we apply **run-length filtering**. Only anomalous sequences exceeding a minimum duration (**min_run**) are flagged as true collective anomalies.



A collective anomaly is defined by the behavior of the group, not the individual point.

Sequence Detector: Custom Rolling MAD

Goal: detect collective anomalies by returning the full interval, not just a representative timestamp. Uses a rolling median baseline and local MAD scaling.

```
hanr_sequence_mad_custom <-  
  function(window_size = 11,  
           z_threshold = 3,  
           min_run = 3) {  
    obj <- harbinger()  
    obj$window_size <- window_size  
    obj$z_threshold <- z_threshold  
    obj$min_run <- min_run  
    class(obj) <- append(  
      "hanr_sequence_mad_custom", class(obj))  
    obj  
  }  
  
detect.hanr_sequence_mad_custom <-  
  function(obj, serie, ...) {  
    obj <- obj$har_store_refs(obj, serie)  
    y <- as.numeric(obj$serie)  
    k <- obj$window_size  
    if (k %% 2 == 0) k <- k + 1  
  
    baseline <- zoo::rollmedian(  
      y, k = k, fill = NA, align = "center")  
    local_mad <- zoo::rollapply(  
      y, width = k,  
      FUN = function(v) stats::mad(v, constant=1),  
      fill = NA, align = "center")
```

```
    baseline <- zoo::na.locf(baseline,  
                             na.rm=FALSE, fromLast=FALSE)  
    baseline <- zoo::na.locf(baseline,  
                             fromLast=TRUE, na.rm=FALSE)  
    local_mad <- zoo::na.locf(local_mad,  
                              na.rm=FALSE, fromLast=FALSE)  
    local_mad <- zoo::na.locf(local_mad,  
                              fromLast=TRUE, na.rm=FALSE)  
    local_mad[local_mad == 0 |  
              is.na(local_mad)] <- 1e-8  
  
    score <- abs(y - baseline) / local_mad  
    raw_flags <- score > obj$z_threshold  
    # ... run grouping (see next slide)  
  }
```

Step by Step

- Ensure `k` is odd for `rollmedian`
- Compute baseline with rolling median (`zoo::rollmedian`)
- Compute local scale with rolling MAD (`zoo::rollapply`)
- Fill edges with `na.locf` (forward and backward)
- Avoid division by zero: replace `MAD=0` with `1e-8`
- Compute robust z-score: $|y - \text{baseline}| / \text{mad}$

 **Key concept:** The local robust z-score is more resistant to outliers than the classical z-score, as it uses median and MAD instead of mean and standard deviation.

Run Grouping and Execution

The core customization: instead of keeping only one point per group, the detector marks the entire contiguous interval as anomalous.

```
# Agrupar runs contíguos de TRUE
idx_true <- which(raw_flags)
runs <- if (length(idx_true) == 0) {
  list(integer(0))
} else {
  split(idx_true,
        cumsum(c(1, diff(idx_true) != 1)))
}
range_flags <- rep(FALSE, length(y))
for (run in runs) {
  if (length(run) >= obj$min_run) {
    range_flags[run] <- TRUE # intervalo completo
  }
}
detection <- obj$har_restore_refs(
  obj, anomalies = range_flags, res = score)
attr(detection, "threshold") <- obj$z_threshold
detection
}
```

```
# Uso
model <- hanr_sequence_mad_custom(
  window_size = 9,
  z_threshold = 2.5,
  min_run = 2)
detection <- detect(model, dataset$serie)
head(detection[detection$event, ])

## idx event  type
## 51  51 TRUE anomaly
## 52  52 TRUE anomaly
```

Step by Step

- Identify TRUE indices with `which(raw_flags)`
- Group contiguous runs with `split + cumsum + diff`
- Filter runs with `length ≥ min_run`
- Mark **all** points in the run as anomalous
- Return detection with `threshold` attribute

📌 **Key concept:** Sequence anomalies are best represented as intervals. Collapsing a segment to a single point hides the duration of the abnormal behavior.

Range-Aware Evaluation

Evaluating collective anomaly detectors requires a nuanced approach that moves beyond traditional point-based metrics. Our framework focuses on range overlap to provide a more accurate assessment of performance.

Problem with Point-Based Metrics

Classical metrics (precision, recall, F1) are inadequate for interval-based anomalies. A detector that flags a single point within a true anomaly interval receives zero precision and recall, even if it accurately identified the anomalous region. This penalizes relevant detections too harshly.

Range-Based Approach

Both detected outputs and ground truth are converted into contiguous intervals (ranges). Evaluation then focuses on the degree of overlap between these detected ranges and the true anomaly ranges.

Range-Based Precision

For each detected range, range-based precision quantifies the fraction that overlaps with any true anomaly range. This metric penalizes detections that are too broad or entirely incorrect.

Range-Based Recall & Alpha

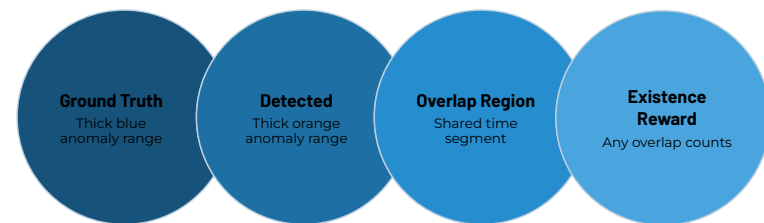
For each true anomaly range, recall determines if it was found. This combines an **existence reward** (was any detection nearby?) and an **overlap reward** (how much of the true range was covered?). The **alpha parameter** ($\alpha \in [0,1]$) controls this balance:

- High alpha rewards existence (finding the anomaly at all).
- Low alpha rewards precise overlap.
- Alpha = 0.5 balances both aspects.

Range-Based F1 Score

The F1 score is calculated as the harmonic mean of range-based precision and recall, providing a single metric that reflects the overall effectiveness of the detector.

Visualizing Range Overlap



This diagram illustrates how range-based evaluation accounts for partial matches. The existence reward acknowledges that an anomaly was found, while precise overlap measures the accuracy of the detected interval.

Range-Aware Evaluator: Support Structures

To evaluate intervals, a custom evaluator is defined that converts boolean vectors into ranges and computes precision/recall based on overlap.

```
har_eval_range_custom <- function(alpha = 0.5) {  
  obj <- daltoolbox::dal_base()  
  obj$alpha <- alpha  
  class(obj) <- append(  
    "har_eval_range_custom", class(obj))  
  obj  
}  
  
range_overlap <-  
  function(a_start, a_end, b_start, b_end) {  
    max(0, min(a_end, b_end) -  
      max(a_start, b_start) + 1)  
  }
```

```
to_ranges <- function(flags) {  
  flags <- as.logical(flags)  
  flags[is.na(flags)] <- FALSE  
  idx <- which(flags)  
  if (length(idx) == 0)  
    return(data.frame(start=integer(0),  
                      end=integer(0),  
                      length=integer(0)))  
  groups <- split(idx,  
    cumsum(c(1, diff(idx) != 1)))  
  starts <- vapply(groups, min, integer(1))  
  ends <- vapply(groups, max, integer(1))  
  data.frame(start=starts, end=ends,  
    length=ends - starts + 1)  
}
```

Step by Step

- Constructor based on `dal_base()` with `alpha` parameter
- `to_ranges()`: converts a logical vector into a `data.frame` of intervals (start, end, length)
- Group contiguous TRUE indices with `split + cumsum`
- `range_overlap()`: computes overlap between two intervals
- Returns 0 if there is no overlap

📌 **Key concept:** `to_ranges()` and `range_overlap()` are reusable helper functions for any range-aware evaluator in Harbinger.

Range-Aware Evaluator: Precision and Recall

The `evaluate()` method combines existence and overlap rewards, controlled by the `alpha` parameter.

```
evaluate.har_eval_range_custom <-  
function(obj, detection, event, ...) {  
  det_ranges <- to_ranges(detection)  
  evt_ranges <- to_ranges(event)  
  
  if (nrow(det_ranges)==0 ||  
      nrow(evt_ranges)==0)  
    return(list(precision=0, recall=0, F1=0))  
  
  recall_scores <- numeric(nrow(evt_ranges))  
  for (i in seq_len(nrow(evt_ranges))) {  
    overlaps <- mapapply(range_overlap,  
      evt_ranges$start[i], evt_ranges$end[i],  
      det_ranges$start, det_ranges$end)  
    best <- max(overlaps)  
    exist <- as.numeric(best > 0)  
    overl <- best / evt_ranges$length[i]  
    recall_scores[i] <-  
      obj$alpha * exist +  
      (1 - obj$alpha) * overl  
  }  
}
```

```
prec_scores <- numeric(nrow(det_ranges))  
for (i in seq_len(nrow(det_ranges))) {  
  overlaps <- mapapply(range_overlap,  
    det_ranges$start[i], det_ranges$end[i],  
    evt_ranges$start, evt_ranges$end)  
  prec_scores[i] <-  
    max(overlaps) / det_ranges$length[i]  
}  
  
precision <- mean(prec_scores)  
recall <- mean(recall_scores)  
F1 <- if ((precision+recall)==0) 0 else  
  2*precision*recall/(precision+recall)  
list(precision=precision,  
     recall=recall, F1=F1)  
}
```

Step by Step

- Convert detection and ground truth to ranges
- **Recall:** for each true range, compute the best overlap with detected ranges
- Combine existence reward (`alpha`) and overlap reward (`1-alpha`)
- **Precision:** for each detected range, fraction that overlaps any true range
- Compute harmonic F1 between precision and recall

❏ **Key concept:** `alpha` controls the trade-off — high values reward finding the range; low values reward precise overlap.

Summary of Extension Points

This section covered collective anomaly detection — flagging full contiguous intervals — and the range-aware evaluator that gives partial credit to detections overlapping the true anomaly window.

Sequence Detector (Rolling MAD)

Contract: constructor + `detect()` returning full intervals.

Example: `hanr_sequence_mad_custom` with `zoo::rollmedian()` + `zoo::rollapply()` for local MAD scaling.

Run Grouping

Contiguous `TRUE` runs are grouped with `split` + `cumsum` + `diff`. Runs with length $\geq$ `min_run` are fully flagged as anomalous intervals.

Range Support Structures

`to_ranges()` converts logical vectors to interval data frames; `range_overlap()` computes overlap between two intervals. Both are reusable helpers.

Range-Aware Precision

For each detected range, the fraction overlapping any true range. Rewards detections that align with ground truth.

Range-Aware Recall

For each true range, combines existence reward (`alpha`) and overlap reward (`1 - alpha`) using the best matching detected range.

Evaluation

Contract: constructor + `evaluate()`.

Example: `har_eval_range_custom` with interval-based precision, recall, and F1.

Additional Sources

Explore the resources below to dive deeper into Harbinger and time series event detection.



CRAN Package

cran.r-project.org/web/packages/harbinger



Event Detection Resources

eic.cefet-rj.br/~eogasawara/en/event-detection-in-time-series



DAL Toolbox

cran.r-project.org/web/packages/daltoolbox



Issues & Discussions

github.com/cefet-rj-dal/harbinger/issues

Open questions, report bugs, or suggest improvements directly in the repository.

References

Ogasawara, E. et al. (2025). *Event Detection in Time Series*. Springer.

Zeileis, A., & Grothendieck, G. (2005). zoo: S3 Infrastructure for Regular and Irregular Time Series. *Journal of Statistical Software*, 14(6), 1–27.
<https://doi.org/10.18637/jss.v014.i06>

Tatbul, N., Lee, T.J., Zdonik, S., Alam, M., & Gottschlich, J. (2018). Precision and Recall for Time Series. *Advances in Neural Information Processing Systems (NeurIPS)*, 31.

Ogasawara, E., Salles, R., Coutinho, R., Pacitti, E., Maseglier, F., Akbarinia, R., & Mattoso, M. (2020). harbinger: An R package for event detection in time series. *SoftwareX*, 12, 100569.

Salles, R., Ogasawara, E., & Pacitti, E. (2022). daltoolbox: An R package for data analytics and learning. *SoftwareX*.